

Harry Myers CS m/46 HW #1 April 21st 2026

HW #1

1. a. $J(w_0, w_1) = \sum_n a_n (w_0 + w_1 x_{n,1} - y_n)^2$

$\frac{\partial J}{\partial w_0} = 2 \sum_n a_n (w_0 + w_1 x_{n,1} - y_n)$, $\frac{\partial J}{\partial w_1} = 2 \sum_n a_n x_{n,1} (w_0 + w_1 x_{n,1} - y_n)$

b. If one weight $a_n \rightarrow \infty$ that means point n is infinitely important to the line must go through k . This completely messes up the optimal solution.

Cont. R2: $J_1(w_0, w_1) = \sum_n a_n (w_0 + w_1 x_{n,1} - y_n)^2 + \lambda (w_0^2 + w_1^2)$

$\frac{\partial J_1}{\partial w_0} = 2 \sum_n a_n (w_0 + w_1 x_{n,1} - y_n) + 2 \lambda w_0$, $\frac{\partial J_1}{\partial w_1} = 2 \sum_n a_n x_{n,1} (w_0 + w_1 x_{n,1} - y_n) + 2 \lambda w_1$

1.C. II. normal equations $\nabla J = 0$ $\theta = (X^T X)^{-1} X^T Y$ comes from minimizing standard loss $J(w) = \frac{1}{2} \|Xw - y\|_2^2$

Add weights a_n $J(w) = \sum_n a_n (w_0 + w_1 x_{n,1} - y_n)^2$ $A = \text{diag}(a_n) = \begin{bmatrix} a_1 & & \\ & \ddots & \\ & & a_n \end{bmatrix}$

$Jw = (Xw - y)^T A (Xw - y)$ (matrix form) $+ \lambda \|w\|^2$

$J_1(w_0, w_1) = \sum_n a_n (w_0 + w_1 x_{n,1} - y_n)^2 + \lambda (w_0^2 + w_1^2)$

$X = \begin{bmatrix} 1 & x_{1,1} \\ 1 & x_{2,1} \\ \vdots & \vdots \\ 1 & x_{n,1} \end{bmatrix}$ $w = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$ $y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$

$\frac{\partial J_1}{\partial w_0} = 2 \sum_n a_n (w_0 + w_1 x_{n,1} - y_n) + 2 \lambda w_0 = 0$ $\frac{\partial J_1}{\partial w_1} = 2 \sum_n a_n x_{n,1} (w_0 + w_1 x_{n,1} - y_n) + 2 \lambda w_1 = 0$

$\begin{bmatrix} 1 & A^T X \\ X^T A & X^T A X \end{bmatrix} w = \begin{bmatrix} X^T A y \\ X^T A X w - X^T A y + \lambda w = 0 \end{bmatrix}$ $X^T A y = w (X^T A X + \lambda I)$

$w^k = (X^T A X + \lambda I)^{-1} X^T A y$

2. logistic loss $h_w(x) = \sigma(w^T x) = \frac{1}{1 + e^{-w^T x}}$

a. when $|w^T x|$ is large $e^{-w^T x} \rightarrow 0$ $\times$ $h_w(x) \rightarrow 1$ or $e^{-w^T x} \rightarrow \infty$ $\times$ $h_w(x) \rightarrow 0$ so always predicts same

b. $h_w(x) = 0.5 = \sigma(w^T x)$ $\frac{1}{2}$ when $e^{-w^T x} = 1$ $\times$ $-w^T x$ is 0 $\times$ $w^T x$ is linear in x

c. $\nabla_w J_{ce} = (\sigma(w^T x) - y) x$ $\nabla_w J_{mse} = (\sigma(w^T x) - y) \sigma'(w^T x) x$ $\sigma'(w^T x) = \left(\frac{1}{1+e^{-x}}\right)' = \frac{e^x}{(e^x + 1)^2} = \frac{1}{(1+e^{-x})^2} = \frac{1}{(1+\sigma(-x))^2}$

$\frac{1}{1+e^{-x}} = 1 - \frac{1}{1+e^x} = \sigma(x) (1 - \sigma(x))$ if $\sigma(w^T x) = 1$ then $\sigma'(w^T x) = 0$

else if $\sigma(w^T x) = 0$ then $\sigma'(w^T x) = 0$ cross entropy gradients: if a model is very wrong it learns fast but if the model is wrong on squared loss it won't learn very much.

d. cross entropy $\rightarrow$ squared loss only affects the training dynamics cause

both loss functions use same model $h(x) = \sigma(w^T x)$ & same threshold to produce some decision boundary

The loss function only affects how fast you get to w . once you have w the boundary looks the same.

3. a. $J(w) = \|xw - y\|_2^2 + \lambda \|w\|_2^2$ $x \in \mathbb{R}^{n \times d}$, $w \in \mathbb{R}^d$, $y \in \mathbb{R}^n$ $\lambda > 0$

$$J(w) = \left\| \begin{bmatrix} x \\ \sqrt{\lambda} I \end{bmatrix} w - \begin{bmatrix} y \\ 0 \end{bmatrix} \right\|_2^2 = \left\| \begin{bmatrix} xw \\ \sqrt{\lambda} w \end{bmatrix} - \begin{bmatrix} y \\ 0 \end{bmatrix} \right\|_2^2 = \|xw - y\|_2^2 + \|\sqrt{\lambda} w\|_2^2$$

$$Jw = \|xw - y\|_2^2 + \lambda \|w\|_2^2 = \|xw - y\|_2^2 + \|\sqrt{\lambda} w\|_2^2 = \left\| \begin{bmatrix} xw - y \\ \sqrt{\lambda} w \end{bmatrix} \right\|_2^2 = \left\| \begin{bmatrix} x \\ \sqrt{\lambda} I \end{bmatrix} w - \begin{bmatrix} y \\ 0 \end{bmatrix} \right\|_2^2$$

3. b. $\frac{1}{\lambda} \quad$ 3 b. I. a data points are being added

$$\text{If each fake data point has only 1 feature } \tilde{x} = \begin{bmatrix} x \\ \sqrt{\lambda} I \end{bmatrix} \begin{bmatrix} y \\ 0 \end{bmatrix}$$

equal to $\sqrt{\lambda}$ & all others are zero & the y for this data point is zero this

means that whenever feature x is active the function reduces that weight. This is meant

to regularize the function & keep the weights smaller.

```
In [4]: import numpy as np
import pandas as pd
df = pd.read_csv('data/question_4_data.csv')
```

```
In [5]: df.head(8)
```

```
Out[5]:
```

	cement	slag	flyash	water	superplasticizer	coarseaggregate	fineaggregate	age	csf
0	540.0	0.0	0.0	162.0	2.5	1040.0	676.0	28	7
1	540.0	0.0	0.0	162.0	2.5	1055.0	676.0	28	6
2	332.5	142.5	0.0	228.0	0.0	932.0	594.0	270	4
3	332.5	142.5	0.0	228.0	0.0	932.0	594.0	365	4
4	198.6	132.4	0.0	192.0	0.0	978.4	825.5	360	4
5	266.0	114.0	0.0	228.0	0.0	932.0	670.0	90	4
6	380.0	95.0	0.0	228.0	0.0	932.0	594.0	365	4
7	380.0	95.0	0.0	228.0	0.0	932.0	594.0	28	3

4.a.

```
In [6]: print("Shape:", df.shape)
print("\nTypes:")
print(df.dtypes)
print("\nNULL elements:\n", df.isnull().sum())
```

Shape: (1030, 9)

Types:

```

cement          float64
slag            float64
flyash          float64
water           float64
superplasticizer float64
coarseaggregate float64
fineaggregate   float64
age             int64
csMPa           float64
dtype: object

```

NULL elements:

```

cement      0
slag        0
flyash      0
water       0
superplasticizer 0
coarseaggregate 0
fineaggregate 0
age         0
csMPa       0
dtype: int64

```

In [7]: `print(df.describe())`

	cement	slag	flyash	water	superplasticizer \
count	1030.000000	1030.000000	1030.000000	1030.000000	1030.000000
mean	281.167864	73.895825	54.188350	181.567282	6.204660
std	104.506364	86.279342	63.997004	21.354219	5.973841
min	102.000000	0.000000	0.000000	121.800000	0.000000
25%	192.375000	0.000000	0.000000	164.900000	0.000000
50%	272.900000	22.000000	0.000000	185.000000	6.400000
75%	350.000000	142.950000	118.300000	192.000000	10.200000
max	540.000000	359.400000	200.100000	247.000000	32.200000

	coarseaggregate	fineaggregate	age	csMPa
count	1030.000000	1030.000000	1030.000000	1030.000000
mean	972.918932	773.580485	45.662136	35.817961
std	77.753954	80.175980	63.169912	16.705742
min	801.000000	594.000000	1.000000	2.330000
25%	932.000000	730.950000	7.000000	23.710000
50%	968.000000	779.500000	28.000000	34.445000
75%	1029.400000	824.000000	56.000000	46.135000
max	1145.000000	992.600000	365.000000	82.600000

There are 1030 samples and no null elements in any of the columns. There are 9 features.

4.1.11

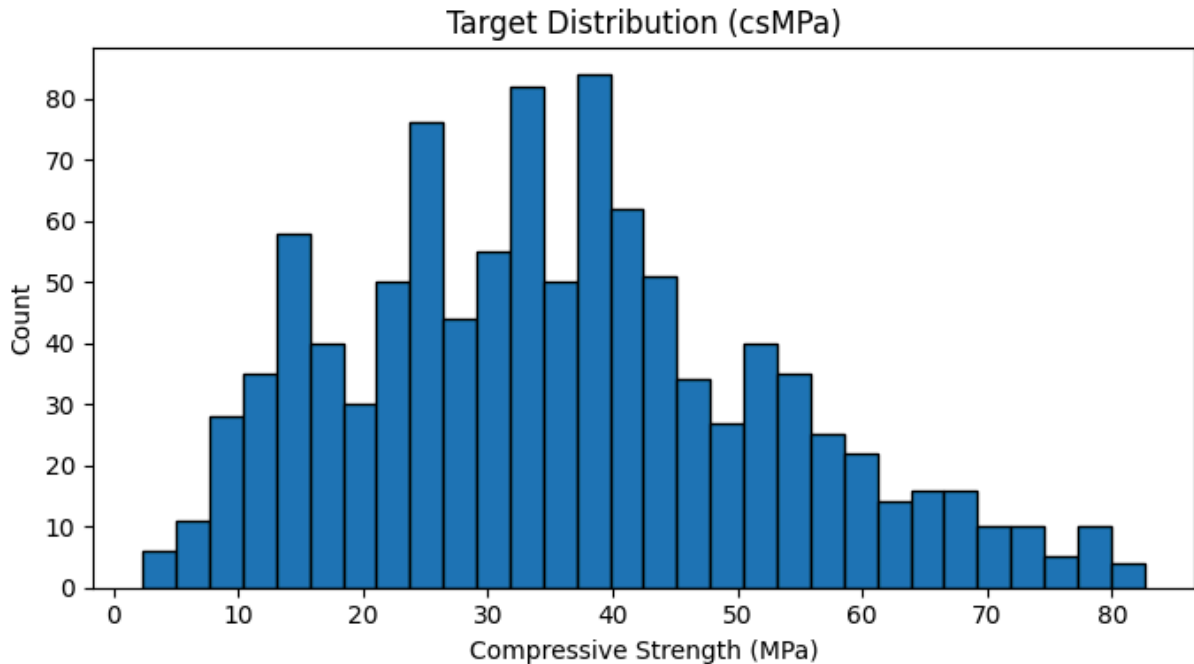
In [8]: `import matplotlib.pyplot as plt`
`import seaborn as sns`

`# (ii) Target distribution`

```

y = df['csMPa']
plt.figure(figsize=(7, 4))
plt.hist(y, bins=30, edgecolor='black')
plt.xlabel('Compressive Strength (MPa)')
plt.ylabel('Count')
plt.title('Target Distribution (csMPa)')
plt.tight_layout()
plt.show()
print(f'Mean: {y.mean():.2f} MPa, Std: {y.std():.2f} MPa')
print('Right-skewed: most values cluster 20-50 MPa with a tail toward higher streng

```



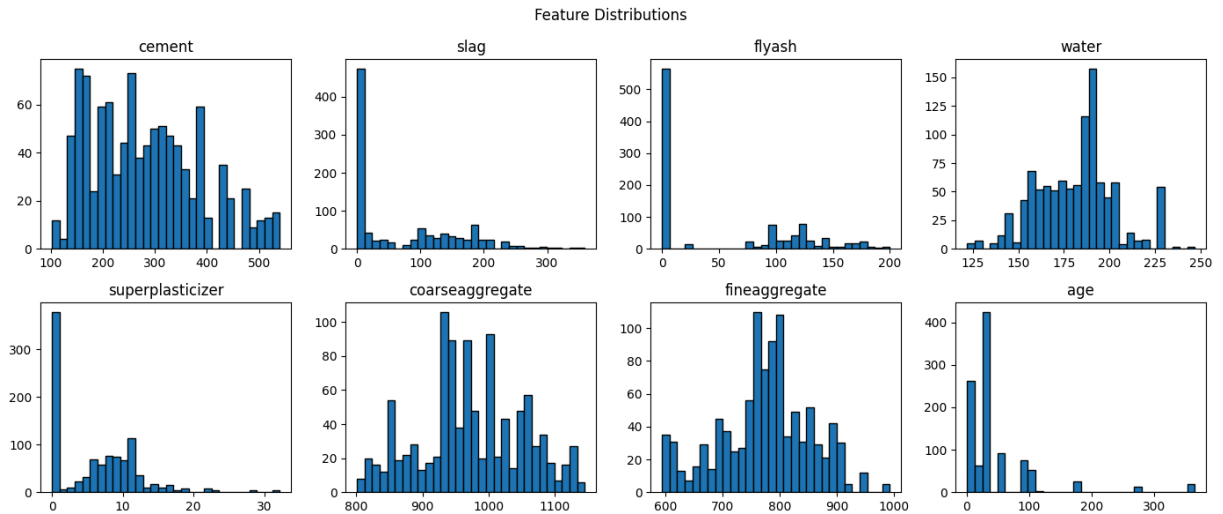
Mean: 35.82 MPa, Std: 16.71 MPa

Right-skewed: most values cluster 20-50 MPa with a tail toward higher strengths.

```

In [9]: # (iii) Feature distributions (2x4 grid)
features = df.columns[:-1].tolist()
fig, axes = plt.subplots(2, 4, figsize=(14, 6))
for ax, feat in zip(axes.flatten(), features):
    ax.hist(df[feat], bins=30, edgecolor='black')
    ax.set_title(feat)
plt.suptitle('Feature Distributions')
plt.tight_layout()
plt.show()
print('slag and flyash spike at zero: absent from many mixtures.')
print('age is right-skewed: most samples cured 28 days.')

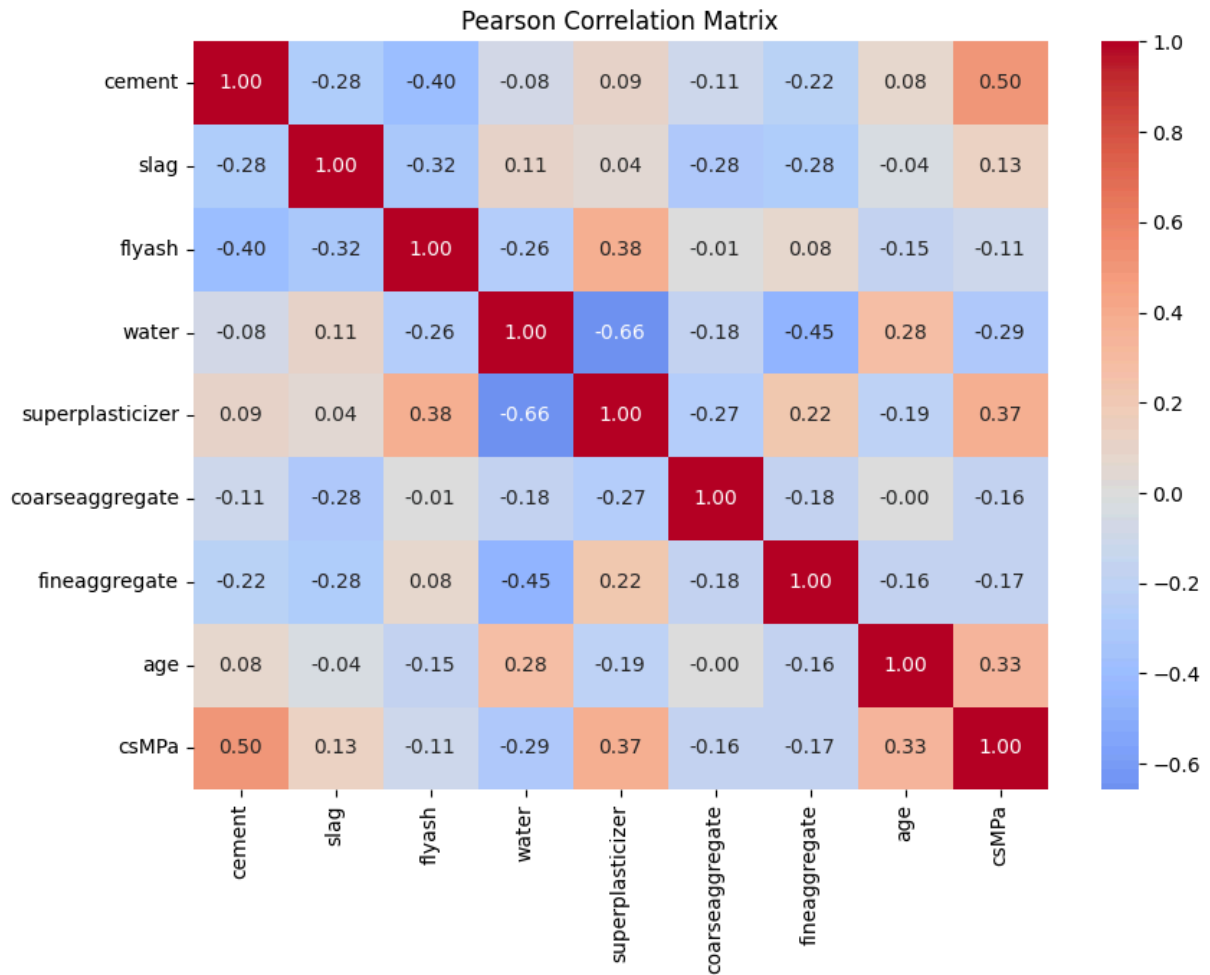
```



slag and flyash spike at zero: absent from many mixtures.

age is right-skewed: most samples cured 28 days.

```
In [10]: # (iv) Pearson correlation heatmap
corr = df.corr()
plt.figure(figsize=(9, 7))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0)
plt.title('Pearson Correlation Matrix')
plt.tight_layout()
plt.show()
target_corr = corr['csMPa'].drop('csMPa').sort_values()
print(f'Most negatively correlated: {target_corr.idxmin()} ({target_corr.min():.2f}')
print(f'Most positively correlated: {target_corr.idxmax()} ({target_corr.max():.2f}')
print('cement/age positively correlated: more cement + longer curing -> stronger co')
print('water negatively correlated: excess water dilutes the cement paste.')
```

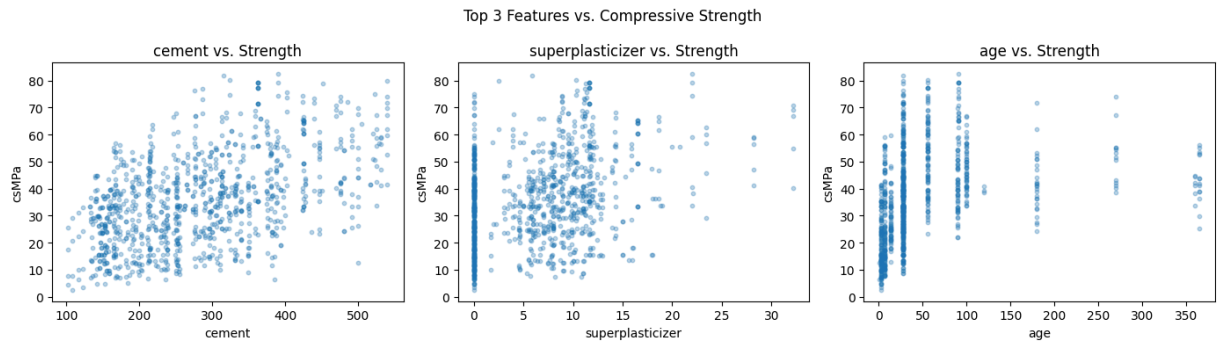
Most negatively correlated: water (-0.29)

Most positively correlated: cement (0.50)

cement/age positively correlated: more cement + longer curing -> stronger concrete.

water negatively correlated: excess water dilutes the cement paste.

```
In [11]: # (v) Scatter plots of top-3 correlated features vs target
top3 = corr['csMPa'].drop('csMPa').abs().nlargest(3).index.tolist()
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
for ax, feat in zip(axes, top3):
    ax.scatter(df[feat], df['csMPa'], alpha=0.3, s=10)
    ax.set_xlabel(feat)
    ax.set_ylabel('csMPa')
    ax.set_title(f'{feat} vs. Strength')
plt.suptitle('Top 3 Features vs. Compressive Strength')
plt.tight_layout()
plt.show()
print(f'Top 3: {top3}')
print('Relationships appear nonlinear/curved')
```



Top 3: ['cement', 'superplasticizer', 'age']

Relationships appear nonlinear/curved

4.b.

```
In [12]: # (i) Train/test split
np.random.seed(42)
X = df.drop(columns='csMPa').values
y = df['csMPa'].values
N = len(X)
idx = np.random.permutation(N)
split = int(0.8 * N)
train_idx, test_idx = idx[:split], idx[split:]
X_train, X_test = X[train_idx], X[test_idx]
y_train, y_test = y[train_idx], y[test_idx]
print(f'Train: {len(X_train)} samples, Test: {len(X_test)} samples')
```

Train: 824 samples, Test: 206 samples

```
In [13]: # (ii) Why standardize
print("""
Why standardize:
    Cement is in hundreds of kg/m^3 while Superplasticizer is below 30 and Age is 1-36
    Fitting directly on raw features means learned weights must compensate for scale
    differences rather than reflecting true feature importance, making them hard to
    interpret. Standardization (z-score) maps every feature to mean=0, std=1 so weigh
    are directly comparable and gradient descent converges faster.""")
```

Why standardize:

Cement is in hundreds of kg/m³ while Superplasticizer is below 30 and Age is 1-36 5 days.

Fitting directly on raw features means learned weights must compensate for scale differences rather than reflecting true feature importance, making them hard to interpret. Standardization (z-score) maps every feature to mean=0, std=1 so weight

s are directly comparable and gradient descent converges faster.

```
In [14]: # (iii) Implement standardization (training stats only)
mu      = X_train.mean(axis=0)
sigma   = X_train.std(axis=0)
X_train_std = (X_train - mu) / sigma
X_test_std  = (X_test - mu) / sigma
print('Using training-set mu/sigma on test set to prevent data leakage.')
```

Using training-set mu/sigma on test set to prevent data leakage.


```
In [15]: # (iv) Verify
print('Train mean (~0):', X_train_std.mean(axis=0).round(4))
print('Train std (~1):', X_train_std.std(axis=0).round(4))
print('Test mean (close to 0):', X_test_std.mean(axis=0).round(4))
print('Test std (close to 1):', X_test_std.std(axis=0).round(4))
print('Test stats differ from exactly 0/1 because we used training-set statistics,'
      ' not test-set statistics, to avoid data leakage.')
```

Train mean (~0): [0. 0. 0. 0. 0. 0. 0. 0.]

Train std (~1): [1. 1. 1. 1. 1. 1. 1. 1.]

Test mean (close to 0): [0.0505 0.0824 -0.0593 0.1463 -0.1321 0.0373 -0.1887 0.0291]

Test std (close to 1): [1.0452 1.1202 0.9925 0.9218 0.894 0.9295 0.9302 0.9496]

Test stats differ from exactly 0/1 because we used training-set statistics, not test-set statistics, to avoid data leakage.

4.c.

```
In [16]: def add_bias(X):
          return np.hstack([np.ones((X.shape[0], 1)), X])

def mse(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

# OLS normal equations:  $w^* = (X^T X)^{-1} X^T y$ 
Xb_train = add_bias(X_train_std)
Xb_test  = add_bias(X_test_std)
w_ols = np.linalg.solve(Xb_train.T @ Xb_train, Xb_train.T @ y_train)

# (i) MSE
ols_train_mse = mse(y_train, Xb_train @ w_ols)
ols_test_mse  = mse(y_test, Xb_test @ w_ols)
print(f'OLS Train MSE: {ols_train_mse:.2f}')
print(f'OLS Test MSE: {ols_test_mse:.2f}')
```

OLS Train MSE: 107.79

OLS Test MSE: 106.86

```
In [17]: # (ii) Largest absolute weight (weights are comparable because features are standard)
feature_names = df.columns[:-1].tolist()
weights = w_ols[1:]
largest_idx = int(np.argmax(np.abs(weights)))
print(f'Largest |weight|: {feature_names[largest_idx]} ({weights[largest_idx]:.4f})')
print('cement typically has the largest weight - it is the primary binder in concrete and has the strongest correlation with compressive strength.')
```

Largest |weight|: cement (13.2180)

cement typically has the largest weight - it is the primary binder in concrete and has the strongest correlation with compressive strength.

4.d.

```
In [18]: from itertools import combinations_with_replacement
          from math import comb

def poly_features(X, degree):
```

```

n, d = X.shape
cols = [np.ones(n)]
for j in range(d):
    cols.append(X[:, j])
if degree >= 2:
    for i, j in combinations_with_replacement(range(d), 2):
        cols.append(X[:, i] * X[:, j])
if degree >= 3:
    for i, j, k in combinations_with_replacement(range(d), 3):
        cols.append(X[:, i] * X[:, j] * X[:, k])
return np.column_stack(cols)

# (i) Feature counts: C(d+p, p)
d = 8
for p in [2, 3]:
    print(f'p={p}: {comb(d+p, p)} features (including bias)')

# (ii) Degree-2
Xp2_train = poly_features(X_train_std, 2)
Xp2_test = poly_features(X_test_std, 2)
w_p2 = np.linalg.solve(Xp2_train.T @ Xp2_train, Xp2_train.T @ y_train)
p2_train_mse = mse(y_train, Xp2_train @ w_p2)
p2_test_mse = mse(y_test, Xp2_test @ w_p2)
print(f'Degree-2 Train MSE: {p2_train_mse:.2f}')
print(f'Degree-2 Test MSE: {p2_test_mse:.2f}')
print('Degree-2 should improve on OLS as squared terms capture nonlinear relationships')

# (iii) Degree-3
Xp3_train = poly_features(X_train_std, 3)
Xp3_test = poly_features(X_test_std, 3)
w_p3 = np.linalg.solve(Xp3_train.T @ Xp3_train, Xp3_train.T @ y_train)
p3_train_mse = mse(y_train, Xp3_train @ w_p3)
p3_test_mse = mse(y_test, Xp3_test @ w_p3)
print(f'Degree-3 Train MSE: {p3_train_mse:.2f}')
print(f'Degree-3 Test MSE: {p3_test_mse:.2f}')
print('Training MSE drops but test MSE likely increases vs degree-2: overfitting.'
      ' C(11,3)=165 features with only ~824 training samples.')

```

p=2: 45 features (including bias)

p=3: 165 features (including bias)

Degree-2 Train MSE: 51.91

Degree-2 Test MSE: 59.68

Degree-2 should improve on OLS as squared terms capture nonlinear relationships.

Degree-3 Train MSE: 19.72

Degree-3 Test MSE: 35.61

Training MSE drops but test MSE likely increases vs degree-2: overfitting. C(11,3)=165 features with only ~824 training samples.

4.e.

```

In [20]: def ridge_fit(X, y, lam):
          d = X.shape[1]
          return np.linalg.solve(X.T @ X + lam * np.eye(d), X.T @ y)

          lambdas = [1e-4, 1e-3, 1e-2, 1e-1, 1, 10, 100]

```

```

# (i) Sweep Lambda
ridge_results = []
print(f"{'lambda':<10} {'Train MSE':>12} {'Test MSE':>12}")
print('-' * 36)
for lam in lambdas:
    w = ridge_fit(Xp3_train, y_train, lam)
    tr = mse(y_train, Xp3_train @ w)
    te = mse(y_test, Xp3_test @ w)
    ridge_results.append((lam, tr, te, w))
    print(f'{'lam':<10} {'tr':>12.2f} {'te':>12.2f}')

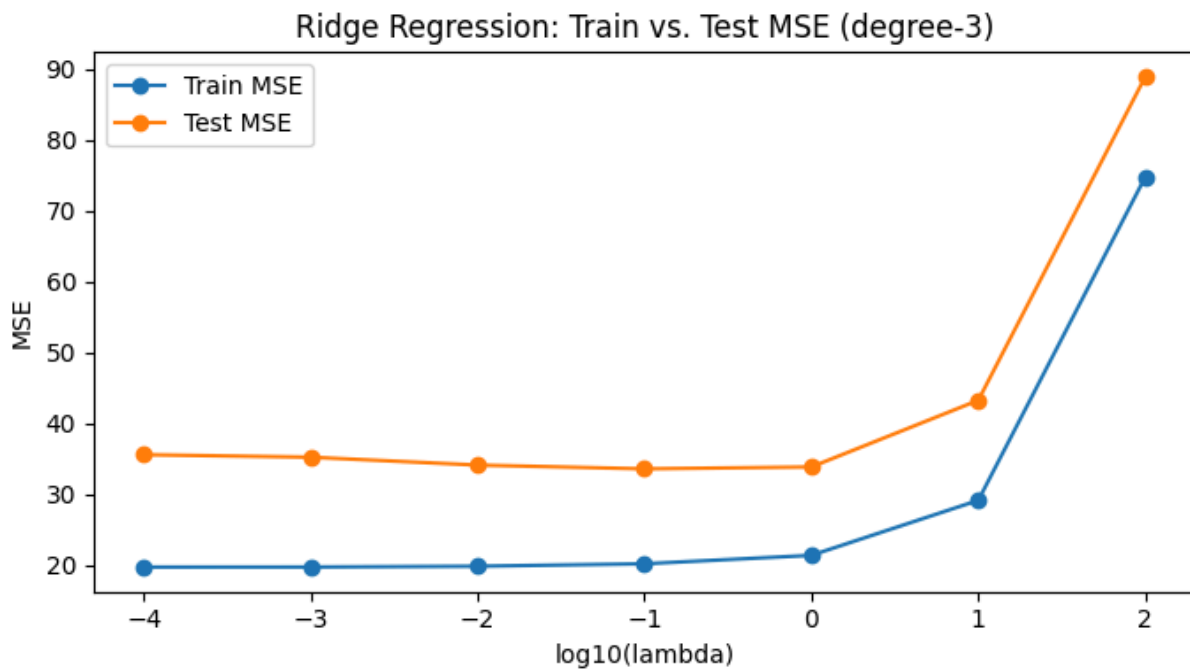
# (ii) Plot train vs test MSE
log_lams = [np.log10(r[0]) for r in ridge_results]
train_mses_r = [r[1] for r in ridge_results]
test_mses_r = [r[2] for r in ridge_results]
plt.figure(figsize=(7, 4))
plt.plot(log_lams, train_mses_r, marker='o', label='Train MSE')
plt.plot(log_lams, test_mses_r, marker='o', label='Test MSE')
plt.xlabel('log10(lambda)')
plt.ylabel('MSE')
plt.title('Ridge Regression: Train vs. Test MSE (degree-3)')
plt.legend()
plt.tight_layout()
plt.show()

# (iii) Best Lambda and comparison
best = min(ridge_results, key=lambda r: r[2])
print(f'Best lambda={best[0]} Train MSE={best[1]:.2f} Test MSE={best[2]:.2f}')
print(f'Unregularized degree-3 test MSE: {p3_test_mse:.2f} -> ridge fixes overfitt')
print(f'Linear OLS test MSE: {ols_test_mse:.2f} -> poly+ridge beats pl')
print('Bias-variance tradeoff: too small lambda = overfitting; too large = underfit')

# (iv) Weight norm vs Lambda
norms = [np.linalg.norm(r[3]) for r in ridge_results]
plt.figure(figsize=(7, 4))
plt.plot(log_lams, norms, marker='o')
plt.xlabel('log10(lambda)')
plt.ylabel('||w*||_2')
plt.title('Ridge Weight Norm vs. lambda')
plt.tight_layout()
plt.show()
print('Weights shrink toward zero as lambda increases.')
print('From Problem 3: ridge = augmented LS with d fake zero-target rows, pulling w

```

lambda	Train MSE	Test MSE
0.0001	19.72	35.56
0.001	19.73	35.21
0.01	19.84	34.10
0.1	20.19	33.57
1	21.38	33.84
10	29.13	43.23
100	74.71	88.84

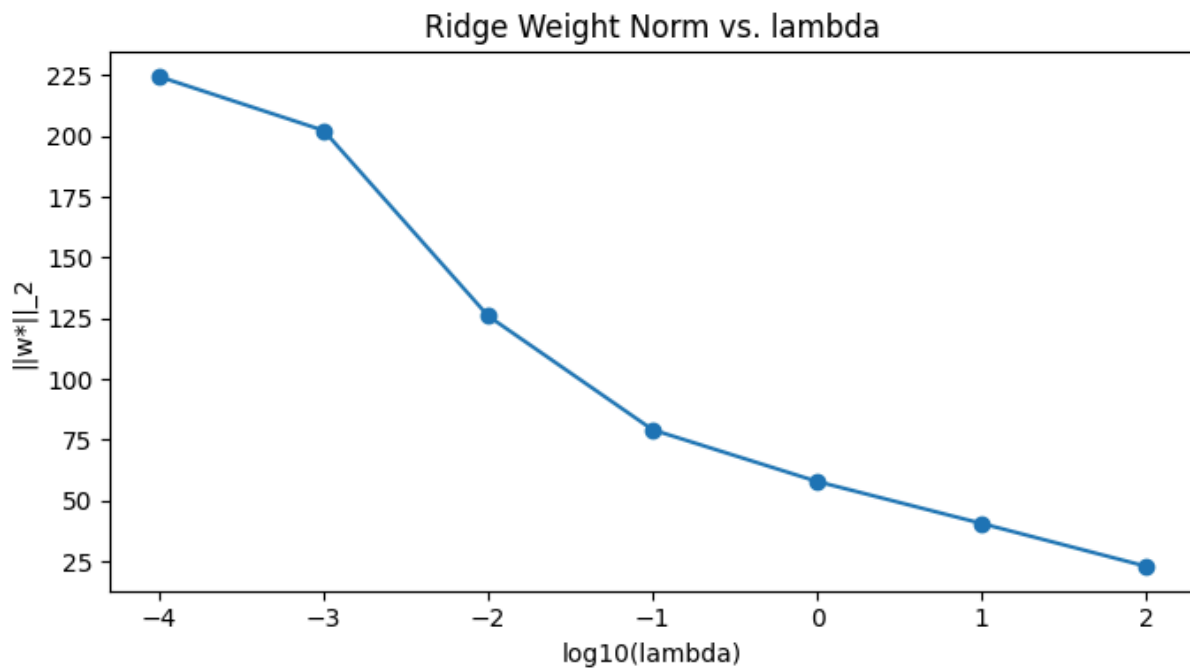


Best $\lambda=0.1$ Train MSE=20.19 Test MSE=33.57

Unregularized degree-3 test MSE: 35.61 $\rightarrow$ ridge fixes overfitting.

Linear OLS test MSE: 106.86 $\rightarrow$ poly+ridge beats plain linear.

Bias-variance tradeoff: too small λ = overfitting; too large = underfitting.



Weights shrink toward zero as λ increases.

From Problem 3: ridge = augmented LS with d fake zero-target rows, pulling weights toward zero.

4.f.

```
In [21]: # (i) 5-fold CV on training data
def kfold_cv(X, y, lambdas, k=5):
    n = len(y)
    fold_size = n // k
    cv_mses = {lam: [] for lam in lambdas}
```



```

for fold in range(k):
    val_idx = np.arange(fold * fold_size, (fold + 1) * fold_size)
    tr_idx = np.concatenate([np.arange(0, fold * fold_size),
                             np.arange((fold + 1) * fold_size, n)])
    Xtr, ytr = X[tr_idx], y[tr_idx]
    Xvl, yvl = X[val_idx], y[val_idx]
    for lam in lambdas:
        w = ridge_fit(Xtr, ytr, lam)
        cv_mses[lam].append(mse(yvl, Xvl @ w))
    return {lam: np.mean(v) for lam, v in cv_mses.items()}

cv_results = kfold_cv(Xp3_train, y_train, lambdas)
print(f"{'lambda':<10} {'Mean Val MSE':>14}")
print('-' * 26)
for lam, val_mse_val in cv_results.items():
    print(f"{'lam':<10} {'val_mse_val':>14.2f}")

# (ii) Best Lambda and final test result
best_lam_cv = min(cv_results, key=cv_results.get)
w_final = ridge_fit(Xp3_train, y_train, best_lam_cv)
final_test_mse = mse(y_test, Xp3_test @ w_final)
print(f'Best lambda from 5-fold CV: {best_lam_cv}')
print(f'Final Test MSE (CV-selected lambda): {final_test_mse:.2f}')
print(f'Single-split best test MSE: {min(r[2] for r in ridge_results):.2f}')
print('CV and single-split conclusions should be similar; CV is more stable.')

# (iii) Why not use test set in CV
print("""
Why not use the test set inside CV:
    Using test-set performance to choose lambda contaminates it - it no longer repres
    unseen data and any reported test MSE becomes optimistically biased. We would eff
    be tuning to the test set. CV uses only training data so the test set remains a c
    estimate of true generalization performance.""")

```

lambda	Mean Val MSE
--------	--------------

0.0001	47.97
0.001	47.68
0.01	46.62
0.1	44.21
1	41.47
10	49.61
100	103.91

Best lambda from 5-fold CV: 1

Final Test MSE (CV-selected lambda): 33.84

Single-split best test MSE: 33.57

CV and single-split conclusions should be similar; CV is more stable.

Why not use the test set inside CV:

Using test-set performance to choose lambda contaminates it - it no longer represents

unseen data and any reported test MSE becomes optimistically biased. We would effectively

be tuning to the test set. CV uses only training data so the test set remains a clean

estimate of true generalization performance.

4.g.

```

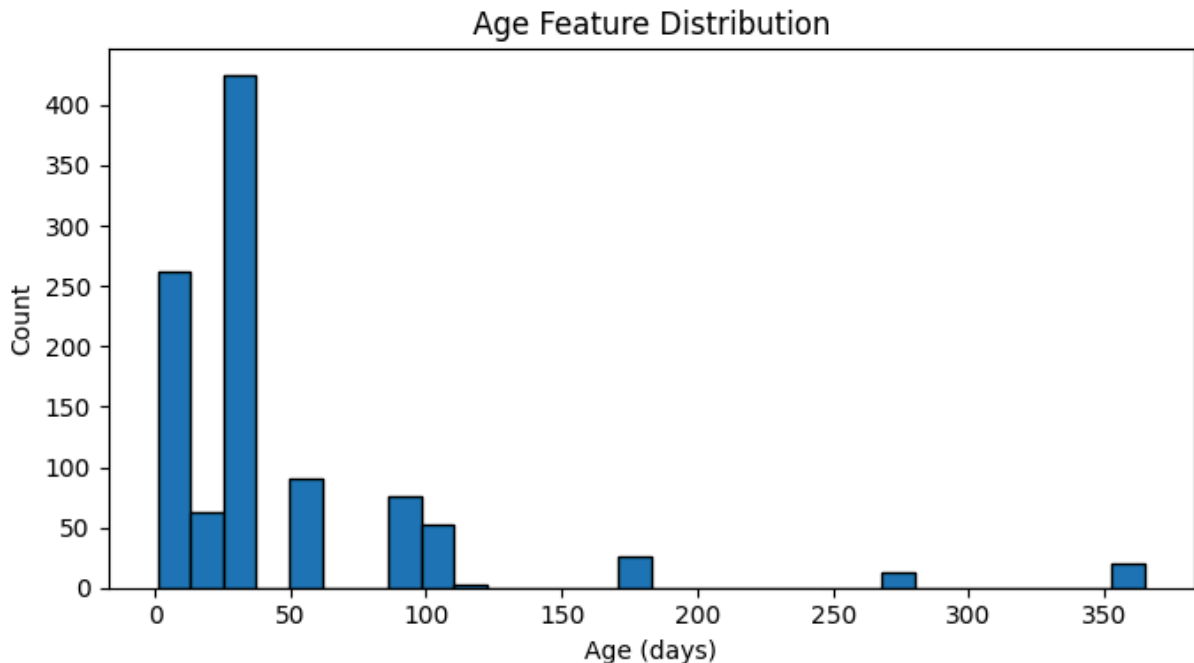
In [22]: # (i) Age histogram
plt.figure(figsize=(7, 4))
plt.hist(df['age'], bins=30, edgecolor='black')
plt.xlabel('Age (days)')
plt.ylabel('Count')
plt.title('Age Feature Distribution')
plt.tight_layout()
plt.show()

distinct_ages = df['age'].nunique()
frac_28 = (df['age'] == 28).mean()
print(f'Distinct age values: {distinct_ages}')
print(f'Fraction with Age=28: {frac_28:.3f}')
print('Heavily right-skewed with a dominant spike at 28 days (industry-standard cur

# (ii) Weighted Least squares with inverse-frequency weights
age_counts = df['age'].value_counts().to_dict()
N_total = len(df)
alphas_all = np.array([N_total / age_counts[df['age'].iloc[i]] for i in range(N_tot
alphas = alphas_all[train_idx]

A = np.diag(alphas)
w_wls = np.linalg.solve(Xb_train.T @ A @ Xb_train, Xb_train.T @ A @ y_train)
wls_train_mse = mse(y_train, Xb_train @ w_wls)
wls_test_mse = mse(y_test, Xb_test @ w_wls)
print(f'WLS Train MSE: {wls_train_mse:.2f}')
print(f'WLS Test MSE: {wls_test_mse:.2f}')
print(f'OLS Train MSE: {ols_train_mse:.2f}')
print(f'OLS Test MSE: {ols_test_mse:.2f}')
print('WLS may have higher overall MSE but gives fairer predictions across all age
      ' A practitioner might prefer it for safety-critical early/late-age predictio

```



Distinct age values: 14

Fraction with Age=28: 0.413

Heavily right-skewed with a dominant spike at 28 days (industry-standard curing time).

WLS Train MSE: 127.82

WLS Test MSE: 125.21

OLS Train MSE: 107.79

OLS Test MSE: 106.86

WLS may have higher overall MSE but gives fairer predictions across all age groups.

A practitioner might prefer it for safety-critical early/late-age predictions.

4.h.

```
In [23]: best_ridge_w = ridge_fit(Xp3_train, y_train, best_lam_cv)

models = [
    ('Linear OLS', mse(y_train, Xb_train @ w_ols), mse(y_test, Xb
    ('Degree-2 polynomial', mse(y_train, Xp2_train @ w_p2), mse(y_test, Xp
    ('Degree-3 polynomial', mse(y_train, Xp3_train @ w_p3), mse(y_test, Xp
    (f'Ridge deg-3 (lam={best_lam_cv})', mse(y_train, Xp3_train @ best_ridge_w), ms
    ('Weighted Linear (WLS)', wls_train_mse, wls_test_mse)
]
print(f"{'Model':<35} {'Train MSE':>12} {'Test MSE':>12}")
print('-' * 61)
for name, tr, te in models:
    print(f"{'name':<35} {'tr':>12.2f} {'te':>12.2f}")

print("""
Discussion:
1. Recommended model: Ridge regression on degree-3 polynomial features.
   It achieves the lowest test MSE by capturing nonlinear ingredient interactions
   while regularization prevents overfitting.

2. Role of regularization: Without it, degree-3 features drastically overfit
   (~165 parameters, ~824 training samples). Ridge shrinks weights toward zero,
   reducing variance at a small bias cost, yielding much better generalization.

3. When to prefer WLS: When reliable predictions are needed across ALL age groups
   Despite higher overall MSE, WLS corrects for sampling imbalance (over-represen
   of 28-day samples). For safety-critical early-age or late-age predictions this
   fairness is worth the small accuracy tradeoff.""")
```

Model	Train MSE	Test MSE

Linear OLS	107.79	106.86
Degree-2 polynomial	51.91	59.68
Degree-3 polynomial	19.72	35.61
Ridge deg-3 ($\lambda=1$)	21.38	33.84
Weighted Linear (WLS)	127.82	125.21

Discussion:

1. Recommended model: Ridge regression on degree-3 polynomial features.
It achieves the lowest test MSE by capturing nonlinear ingredient interactions while regularization prevents overfitting.
2. Role of regularization: Without it, degree-3 features drastically overfit (~165 parameters, ~824 training samples). Ridge shrinks weights toward zero, reducing variance at a small bias cost, yielding much better generalization.
3. When to prefer WLS: When reliable predictions are needed across ALL age groups. Despite higher overall MSE, WLS corrects for sampling imbalance (over-representation of 28-day samples). For safety-critical early-age or late-age predictions this fairness is worth the small accuracy tradeoff.

```
In [10]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('data/question_5_data/train.csv')
```

5.a.

```
In [11]: # (i) First Look
print('Shape:', df.shape)
print('\nColumns:', df.columns.tolist())
print('\nData types:')
print(df.dtypes)
num_cols_all = df.select_dtypes(include='number').columns.tolist()
cat_cols_all = df.select_dtypes(exclude='number').columns.tolist()
print(f'\nNumerical: {len(num_cols_all)} -> {num_cols_all}')
print(f'\nCategorical/other: {len(cat_cols_all)} -> {cat_cols_all}')
df.head()
```

Shape: (8693, 14)

Columns: ['PassengerId', 'HomePlanet', 'CryoSleep', 'Cabin', 'Destination', 'Age', 'VIP', 'RoomService', 'FoodCourt', 'ShoppingMall', 'Spa', 'VRDeck', 'Name', 'Transported']

Data types:

PassengerId	str
HomePlanet	str
CryoSleep	object
Cabin	str
Destination	str
Age	float64
VIP	object
RoomService	float64
FoodCourt	float64
ShoppingMall	float64
Spa	float64
VRDeck	float64
Name	str
Transported	bool

dtype: object

Numerical: 6 -> ['Age', 'RoomService', 'FoodCourt', 'ShoppingMall', 'Spa', 'VRDeck']
 Categorical/other: 8 -> ['PassengerId', 'HomePlanet', 'CryoSleep', 'Cabin', 'Destination', 'VIP', 'Name', 'Transported']

Out[11]:

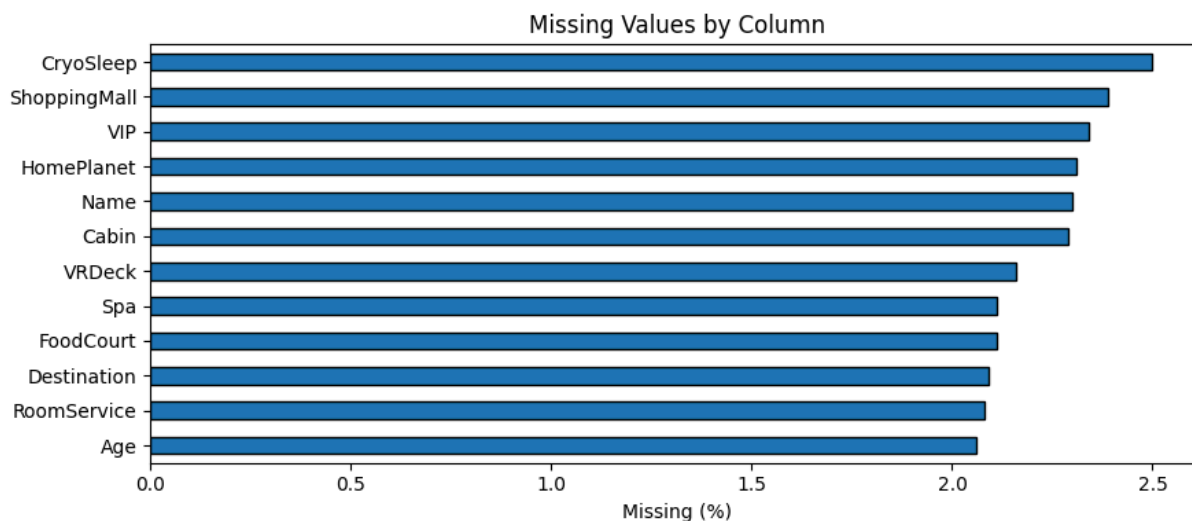
	PassengerId	HomePlanet	CryoSleep	Cabin	Destination	Age	VIP	RoomService	Foc
0	0001_01	Europa	False	B/0/P	TRAPPIST-1e	39.0	False	0.0	
1	0002_01	Earth	False	F/0/S	TRAPPIST-1e	24.0	False	109.0	
2	0003_01	Europa	False	A/0/S	TRAPPIST-1e	58.0	True	43.0	
3	0003_02	Europa	False	A/0/S	TRAPPIST-1e	33.0	False	0.0	
4	0004_01	Earth	False	F/1/S	TRAPPIST-1e	16.0	False	303.0	

In [12]:

```
# (ii) Missing values
missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(2)
miss_df = pd.DataFrame({'Missing Count': missing, 'Missing %': missing_pct})
print(miss_df.sort_values('Missing Count', ascending=False).to_string())

miss_pct_nonzero = missing_pct[missing_pct > 0].sort_values()
plt.figure(figsize=(9, 4))
miss_pct_nonzero.plot(kind='barh', edgecolor='black')
plt.xlabel('Missing (%)')
plt.title('Missing Values by Column')
plt.tight_layout()
plt.show()
print('Missing values are spread relatively evenly (~2%) across most columns.')
print('No single column dominates, so simple imputation (mode/mean) is reasonable.'
```

	Missing Count	Missing %
CryoSleep	217	2.50
ShoppingMall	208	2.39
VIP	203	2.34
HomePlanet	201	2.31
Name	200	2.30
Cabin	199	2.29
VRDeck	188	2.16
Spa	183	2.11
FoodCourt	183	2.11
Destination	182	2.09
RoomService	181	2.08
Age	179	2.06
PassengerId	0	0.00
Transported	0	0.00

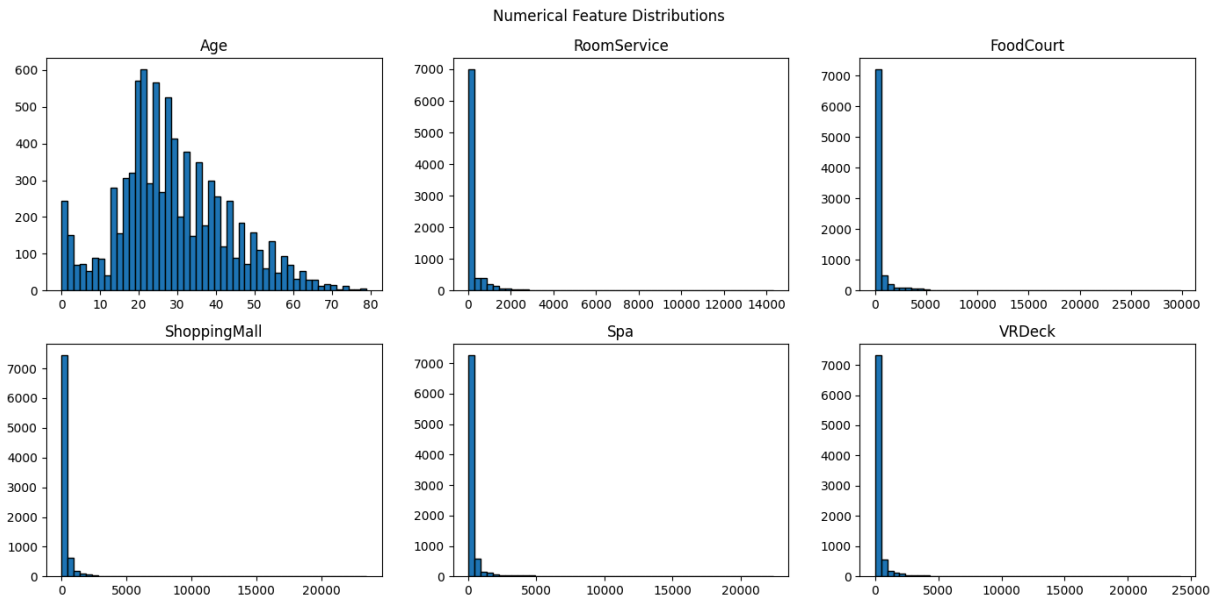


Missing values are spread relatively evenly (~2%) across most columns.
No single column dominates, so simple imputation (mode/mean) is reasonable.

```
In [13]: # (iii) Target balance
transported_col = df['Transported'].astype(int)
frac = transported_col.mean()
print(f'Transported: {frac:.3f} ({transported_col.sum()} / {len(transported_col)})')
print('Dataset is roughly balanced (~50/50), so accuracy is a meaningful metric.')
```

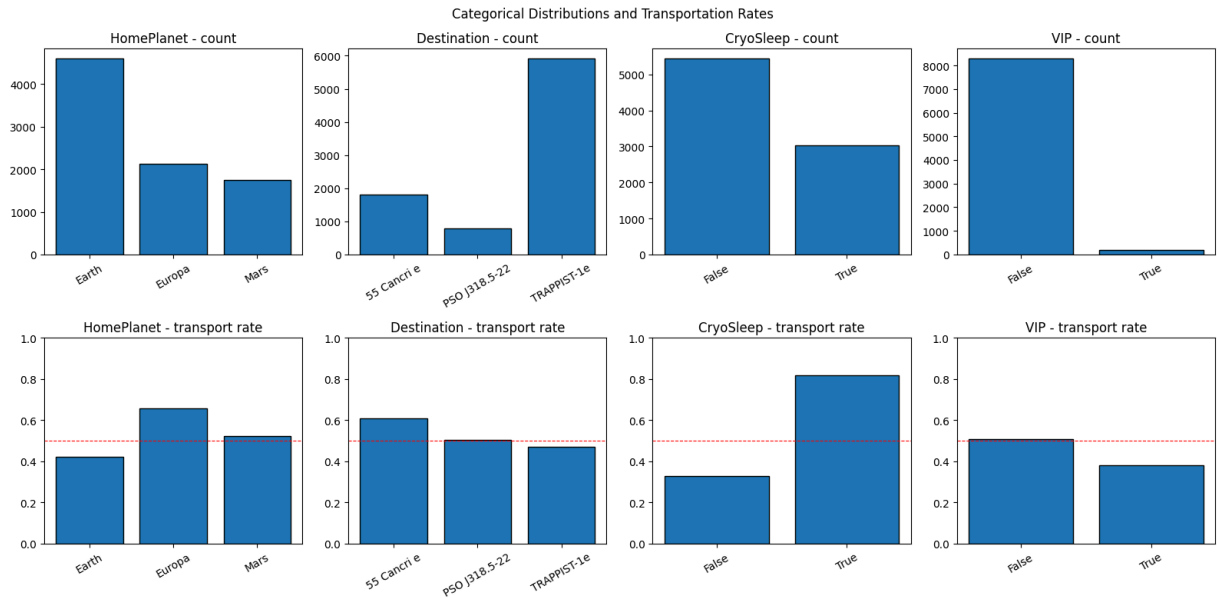
Transported: 0.504 (4378 / 8693)
Dataset is roughly balanced (~50/50), so accuracy is a meaningful metric.

```
In [14]: # (iv) Numerical feature distributions
num_feats = ['Age', 'RoomService', 'FoodCourt', 'ShoppingMall', 'Spa', 'VRDeck']
fig, axes = plt.subplots(2, 3, figsize=(14, 7))
for ax, col in zip(axes.flatten(), num_feats):
    ax.hist(df[col].dropna(), bins=50, edgecolor='black')
    ax.set_title(col)
plt.suptitle('Numerical Feature Distributions')
plt.tight_layout()
plt.show()
print('RoomService, FoodCourt, ShoppingMall, Spa, VRDeck are heavily right-skewed.')
print('The vast majority of passengers have spending = 0; a few outliers spend thou')
print('This motivates the log-transform and zero-spending indicator in part (e).')
```



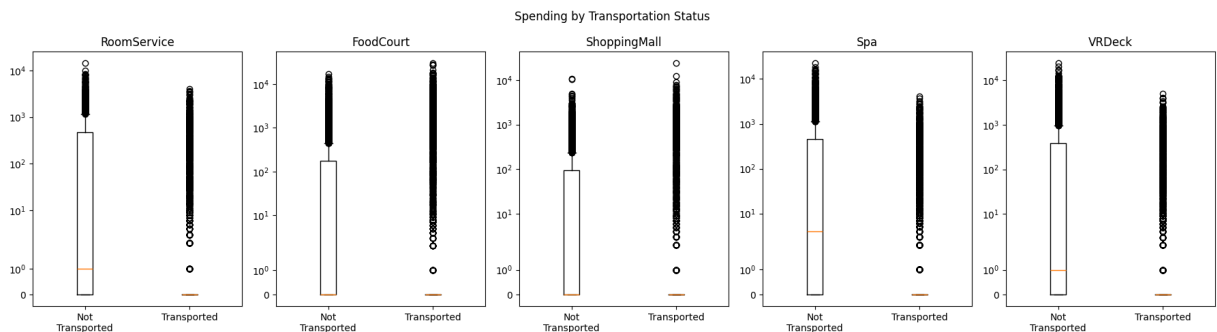
RoomService, FoodCourt, ShoppingMall, Spa, VRDeck are heavily right-skewed. The vast majority of passengers have spending = 0; a few outliers spend thousands. This motivates the log-transform and zero-spending indicator in part (e).

```
In [15]: # (v) Categorical feature distributions + transportation rate
cat_feats = ['HomePlanet', 'Destination', 'CryoSleep', 'VIP']
fig, axes = plt.subplots(2, 4, figsize=(16, 8))
df_t = df.copy()
df_t['Transported'] = df_t['Transported'].astype(int)
for i, col in enumerate(cat_feats):
    counts = df_t[col].value_counts().sort_index()
    axes[0, i].bar(counts.index.astype(str), counts.values, edgecolor='black')
    axes[0, i].set_title(f'{col} - count')
    axes[0, i].tick_params(axis='x', rotation=30)
    rate = df_t.groupby(col)['Transported'].mean().sort_index()
    axes[1, i].bar(rate.index.astype(str), rate.values, edgecolor='black')
    axes[1, i].set_title(f'{col} - transport rate')
    axes[1, i].set_ylim(0, 1)
    axes[1, i].axhline(0.5, color='red', linestyle='--', linewidth=0.8)
    axes[1, i].tick_params(axis='x', rotation=30)
plt.suptitle('Categorical Distributions and Transportation Rates')
plt.tight_layout()
plt.show()
print('CryoSleep is most predictive: CryoSleep=True -> ~80% transported vs ~40% for')
print('This makes physical sense: cryosleep passengers cannot visit amenities and a')
print('likely to have been transported to the alternate dimension.')
```



CryoSleep is most predictive: CryoSleep=True -> ~80% transported vs ~40% for False. This makes physical sense: cryosleep passengers cannot visit amenities and are more likely to have been transported to the alternate dimension.

```
In [16]: # (vi) Spending features vs Transported (box plots)
spend_feats = ['RoomService', 'FoodCourt', 'ShoppingMall', 'Spa', 'VRDeck']
df_t = df.copy()
df_t['Transported'] = df_t['Transported'].astype(int)
fig, axes = plt.subplots(1, 5, figsize=(18, 5))
for ax, col in zip(axes, spend_feats):
    d0 = df_t[df_t['Transported'] == 0][col].dropna()
    d1 = df_t[df_t['Transported'] == 1][col].dropna()
    ax.boxplot([d0, d1], tick_labels=['Not\nTransported', 'Transported'])
    ax.set_title(col)
    ax.set_yscale('symlog')
plt.suptitle('Spending by Transportation Status')
plt.tight_layout()
plt.show()
print('Transported passengers tend to spend LESS on all amenities.')
print('Hypothesis: passengers in CryoSleep (strongly associated with being transported) cannot physically access amenities, so their spending is $0.')
```



Transported passengers tend to spend LESS on all amenities.
Hypothesis: passengers in CryoSleep (strongly associated with being transported) cannot physically access amenities, so their spending is \$0.

5.b.

```

In [17]: # (i) Drop uninformative columns
df_work = df.drop(columns=['PassengerId', 'Name', 'Cabin'])
print('Dropped PassengerId: unique identifier, no predictive content.')
print('Dropped Name: free-text string requiring NLP to use.')
print('Dropped Cabin: has exploitable structure (deck/num/side) but parsing is out

# (v) Train/val/test split BEFORE imputation to prevent data Leakage
# Strategy: 80/20 -> (train+val) vs test, then 75/25 -> train vs val => 60/20/20 ov
np.random.seed(42)
N = len(df_work)
idx = np.random.permutation(N)
test_cut = int(0.8 * N)
trainval_idx = idx[:test_cut]
test_idx = idx[test_cut:]
val_cut = int(0.75 * len(trainval_idx))
train_idx = trainval_idx[:val_cut]
val_idx = trainval_idx[val_cut:]
print(f'\nTrain: {len(train_idx)}, Val: {len(val_idx)}, Test: {len(test_idx)}')

df_t2 = df_work.copy()
df_t2['Transported'] = df_t2['Transported'].astype(int)
print(f'Train transported: {df_t2.iloc[train_idx]["Transported"].mean():.3f}')
print(f'Val transported: {df_t2.iloc[val_idx]["Transported"].mean():.3f}')
print(f'Test transported: {df_t2.iloc[test_idx]["Transported"].mean():.3f}')
print('Class balance is approximately preserved across all three splits.')

# Separate into train/val/test dataframes
df_train = df_work.iloc[train_idx].copy().reset_index(drop=True)
df_val = df_work.iloc[val_idx].copy().reset_index(drop=True)
df_test = df_work.iloc[test_idx].copy().reset_index(drop=True)

cat_cols = ['HomePlanet', 'Destination', 'CryoSleep', 'VIP']
num_cols = ['Age', 'RoomService', 'FoodCourt', 'ShoppingMall', 'Spa', 'VRDeck']

# (ii) Fill categorical NaNs with mode from training set
cat_modes = {col: df_train[col].mode()[0] for col in cat_cols}
for split in [df_train, df_val, df_test]:
    for col, mode_val in cat_modes.items():
        split[col] = split[col].fillna(mode_val)
print('\nCategorical modes:', cat_modes)

# (iii) Fill numerical NaNs with mean from training set
num_means = {col: df_train[col].mean() for col in num_cols}
for split in [df_train, df_val, df_test]:
    for col, mean_val in num_means.items():
        split[col] = split[col].fillna(mean_val)
print('Numerical means computed on training set and applied to all splits.')

# (iv) Encode categorical variables
# CryoSleep and VIP: boolean -> 0/1
for split in [df_train, df_val, df_test]:
    split['CryoSleep'] = split['CryoSleep'].astype(int)
    split['VIP'] = split['VIP'].astype(int)

# HomePlanet and Destination: one-hot with drop_first=True (k-1 dummies to avoid mu

```

```

oh_cols = ['HomePlanet', 'Destination']
df_train = pd.get_dummies(df_train, columns=oh_cols, drop_first=True)
df_val = pd.get_dummies(df_val, columns=oh_cols, drop_first=True)
df_test = pd.get_dummies(df_test, columns=oh_cols, drop_first=True)

# Align columns (in case val/test missing a rare category)
feature_cols = [c for c in df_train.columns if c != 'Transported']
df_val = df_val.reindex(columns=df_train.columns, fill_value=0)
df_test = df_test.reindex(columns=df_train.columns, fill_value=0)

print(f'\nTotal features after encoding: {len(feature_cols)}')
print('Features:', feature_cols)

# (vi) Feature standardization using training set stats
num_feat_cols = [c for c in feature_cols if c in num_cols]
mu = df_train[num_feat_cols].mean()
sigma = df_train[num_feat_cols].std().replace(0, 1)
for split in [df_train, df_val, df_test]:
    split[num_feat_cols] = (split[num_feat_cols] - mu) / sigma

# Extract arrays
def to_arrays(df_split, feat_cols):
    X = df_split[feat_cols].values.astype(float)
    y = df_split['Transported'].astype(int).values
    return X, y

X_train, y_train = to_arrays(df_train, feature_cols)
X_val, y_val = to_arrays(df_val, feature_cols)
X_test, y_test = to_arrays(df_test, feature_cols)

# Add bias column
def add_bias(X):
    return np.hstack([np.ones((X.shape[0], 1)), X])

Xb_train = add_bias(X_train)
Xb_val = add_bias(X_val)
Xb_test = add_bias(X_test)
print(f'\nDesign matrix shape - Train: {Xb_train.shape}, Val: {Xb_val.shape}, Test: {Xb_test.shape}')
print('\nFirst 5 rows of processed training matrix:')
print(pd.DataFrame(Xb_train[:5], columns=['bias'] + feature_cols).to_string())

```


Dropped PassengerId: unique identifier, no predictive content.

Dropped Name: free-text string requiring NLP to use.

Dropped Cabin: has exploitable structure (deck/num/side) but parsing is out of scope.

Train: 5215, Val: 1739, Test: 1739

Train transported: 0.504

Val transported: 0.497

Test transported: 0.510

Class balance is approximately preserved across all three splits.

Categorical modes: {'HomePlanet': 'Earth', 'Destination': 'TRAPPIST-1e', 'CryoSleep': False, 'VIP': False}

Numerical means computed on training set and applied to all splits.

Total features after encoding: 12

Features: ['CryoSleep', 'Age', 'VIP', 'RoomService', 'FoodCourt', 'ShoppingMall', 'Spa', 'VRDeck', 'HomePlanet_Europa', 'HomePlanet_Mars', 'Destination_PSO J318.5-22', 'Destination_TRAPPIST-1e']

Design matrix shape - Train: (5215, 13), Val: (1739, 13), Test: (1739, 13)

First 5 rows of processed training matrix:

	bias	CryoSleep	Age	VIP	RoomService	FoodCourt	ShoppingMall	Spa
	VRDeck	HomePlanet_Europa	HomePlanet_Mars	Destination_PSO J318.5-22	Destination_TRAPPIST-1e			
0	1.0	0.0	-0.683979	0.0	0.319974	-0.067037	0.729398	-2.697173e-0
1	0.656499		0.0		1.0		0.0	
1.0								
1	1.0	0.0	-0.753845	0.0	-0.338192	0.286851	-0.282392	-2.723118e-0
1	-0.270895		0.0		0.0		0.0	
1.0								
2	1.0	1.0	0.853069	0.0	-0.344566	-0.289572	-0.282392	-2.723118e-0
1	-0.271773		0.0		0.0		0.0	
1.0								
3	1.0	0.0	0.433874	0.0	-0.344566	-0.074051	0.413413	4.916008e-1
7	-0.271773		0.0		0.0		0.0	
1.0								
4	1.0	1.0	0.992800	0.0	-0.344566	-0.289572	-0.282392	-2.723118e-0
1	-0.271773		1.0		0.0		0.0	
1.0								

5.c.

```
In [18]: def sigmoid(z):
          return 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))

          def ce_loss(X, y, w):
              p = sigmoid(X @ w)
              p = np.clip(p, 1e-12, 1 - 1e-12)
              return -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))

          def ce_grad(X, y, w):
              return X.T @ (sigmoid(X @ w) - y) / len(y)

          def accuracy(X, y, w, thresh=0.5):
```

```

return np.mean((sigmoid(X @ w) >= thresh).astype(int) == y)

# (i) Gradient: grad J_CE = (1/N) X^T (sigma(Xw) - y)
# This matches Problem 2(c): gradient for one example is (sigma(w^T x) - y) x.
print('grad J_CE = (1/N) * X^T * (sigma(Xw) - y)')
print('Each row contributes (sigma(w^T x_i) - y_i) * x_i, averaged over N samples.'

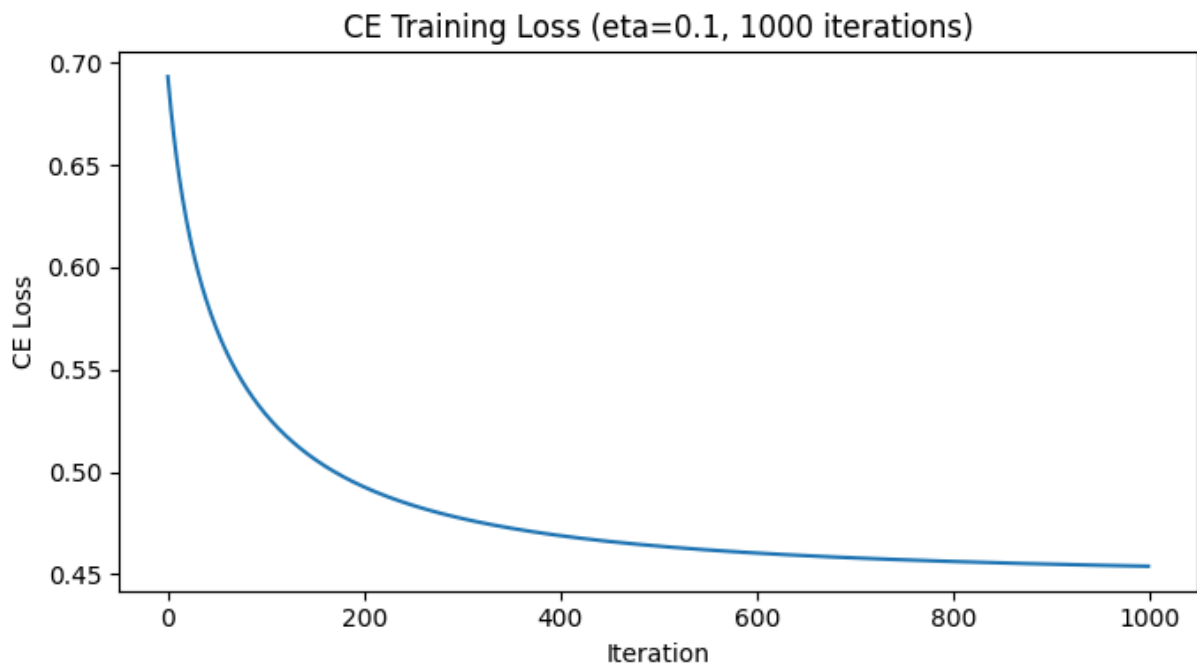
# (ii) Gradient descent, eta=0.1, 1000 iterations
eta = 0.1
n_iter = 1000
w_ce = np.zeros(Xb_train.shape[1])
ce_losses = []
for t in range(n_iter):
    ce_losses.append(ce_loss(Xb_train, y_train, w_ce))
    w_ce -= eta * ce_grad(Xb_train, y_train, w_ce)

plt.figure(figsize=(7, 4))
plt.plot(ce_losses)
plt.xlabel('Iteration')
plt.ylabel('CE Loss')
plt.title('CE Training Loss (eta=0.1, 1000 iterations)')
plt.tight_layout()
plt.show()
print(f'Initial CE loss: {ce_losses[0]:.4f}')
print(f'Final CE loss: {ce_losses[-1]:.4f}')
print('Loss converges smoothly - cross-entropy gradient remains large even for conf

# (iii) Train and validation accuracy
train_acc_ce = accuracy(Xb_train, y_train, w_ce)
val_acc_ce = accuracy(Xb_val, y_val, w_ce)
print(f'\nCE Train accuracy: {train_acc_ce:.4f}')
print(f'CE Val accuracy: {val_acc_ce:.4f}')

```

grad J_CE = (1/N) * X^T * (sigma(Xw) - y)
Each row contributes (sigma(w^T x_i) - y_i) * x_i, averaged over N samples.



Initial CE loss: 0.6931

Final CE loss: 0.4539

Loss converges smoothly - cross-entropy gradient remains large even for confident wrong predictions.

CE Train accuracy: 0.7806

CE Val accuracy: 0.7878

5.d.

```
In [19]: def mse_loss(X, y, w):
    p = sigmoid(X @ w)
    return np.mean((p - y) ** 2)

def mse_grad(X, y, w):
    p = sigmoid(X @ w)
    sp = p * (1 - p) # sigmoid derivative sigma'(z)
    return X.T @ (2 * (p - y) * sp) / len(y)

# (i) Gradient: grad J_MSE = (2/N) X^T [(sigma(Xw) - y) * sigma'(Xw)]
# Extra factor vs CE: sigma'(w^T x) = sigma(w^T x)(1 - sigma(w^T x))
# When the model is confidently wrong (sigma ~ 0 or 1), sigma' ~ 0 -> vanishing gradient
print('grad J_MSE = (2/N) * X^T * [(sigma(Xw) - y) * sigma(Xw) * (1 - sigma(Xw))]\n')
print('Extra factor sigma(z)(1-sigma(z)) vs CE - this vanishes when model is confident\n')
print('causing the vanishing-gradient effect for MSE loss.')

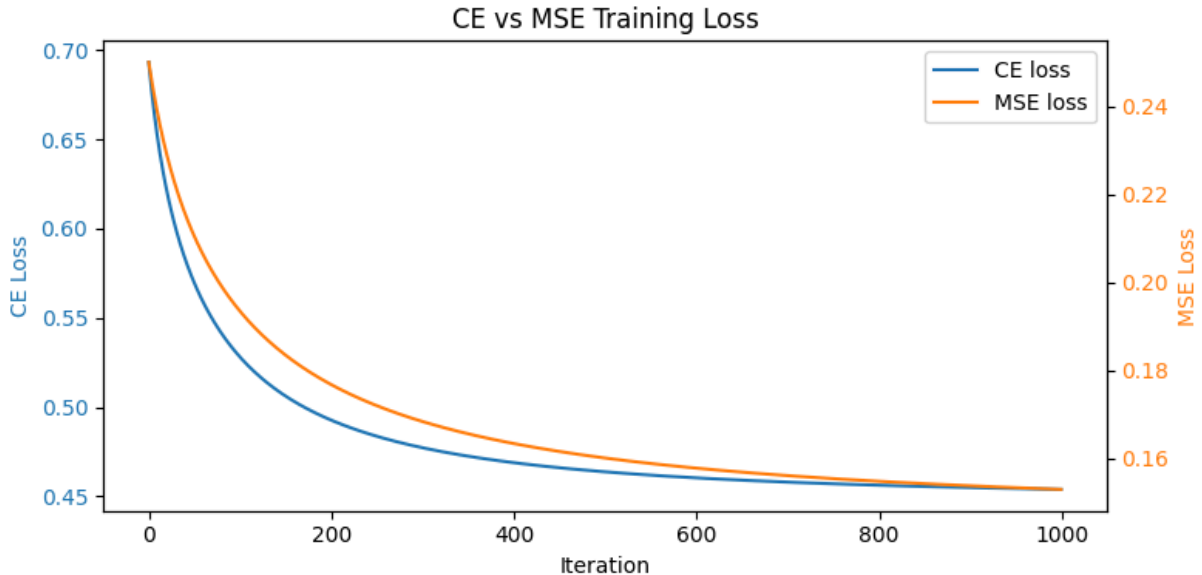
# (ii) GD for MSE, same eta and iterations; plot both loss curves
w_mse = np.zeros(Xb_train.shape[1])
mse_losses = []
for t in range(n_iter):
    mse_losses.append(mse_loss(Xb_train, y_train, w_mse))
    w_mse -= eta * mse_grad(Xb_train, y_train, w_mse)

fig, ax1 = plt.subplots(figsize=(8, 4))
ax2 = ax1.twinx()
ax1.plot(ce_losses, color='tab:blue', label='CE loss')
ax2.plot(mse_losses, color='tab:orange', label='MSE loss')
ax1.set_xlabel('Iteration')
ax1.set_ylabel('CE Loss', color='tab:blue')
ax2.set_ylabel('MSE Loss', color='tab:orange')
ax1.tick_params(axis='y', labelcolor='tab:blue')
ax2.tick_params(axis='y', labelcolor='tab:orange')
lines = [plt.Line2D([0],[0],color='tab:blue'),plt.Line2D([0],[0],color='tab:orange')]
ax1.legend(lines, ['CE loss','MSE loss'], loc='upper right')
plt.title('CE vs MSE Training Loss')
plt.tight_layout()
plt.show()

# (iii) Train/val accuracy for MSE
train_acc_mse = accuracy(Xb_train, y_train, w_mse)
val_acc_mse = accuracy(Xb_val, y_val, w_mse)
print(f'MSE Train accuracy: {train_acc_mse:.4f}')
print(f'MSE Val accuracy: {val_acc_mse:.4f}')
print(f'CE Train accuracy: {train_acc_ce:.4f}')
print(f'CE Val accuracy: {val_acc_ce:.4f}')
print('\nCE typically outperforms MSE for classification.')
```

```
print('When the model is confidently wrong, MSE sigma-prime -> 0, nearly halting le
print('CE does not have this issue - its gradient stays large regardless of confide
```

grad $J_{\text{MSE}} = (2/N) * X^T * [(\text{sigma}(Xw) - y) * \text{sigma}(Xw) * (1 - \text{sigma}(Xw))]$
 Extra factor $\text{sigma}(z)(1-\text{sigma}(z))$ vs CE - this vanishes when model is confident, causing the vanishing-gradient effect for MSE loss.



MSE Train accuracy: 0.7843
 MSE Val accuracy: 0.7861
 CE Train accuracy: 0.7806
 CE Val accuracy: 0.7878

CE typically outperforms MSE for classification.

When the model is confidently wrong, MSE $\text{sigma-prime} \rightarrow 0$, nearly halting learning.
 CE does not have this issue - its gradient stays large regardless of confidence.

5.e.

```
In [20]: from itertools import combinations_with_replacement

spend_raw = ['RoomService', 'FoodCourt', 'ShoppingMall', 'Spa', 'VRDeck']

# (i) Zero-spending indicator + log(1 + total_spending)
def add_spend_features(df_split, train_log_mu=None, train_log_sigma=None):
    s = df_split[spend_raw]
    # Un-standardize to get raw spending back (mu and sigma from original training)
    # Actually we'll compute on the un-standardized versions separately - store raw
    raise NotImplementedError # placeholder; see below

# Re-build from scratch with raw spending stored
# Redo preprocessing keeping raw spending columns before standardization
spend_feats_raw = {}
for name, split_idx in [('train', train_idx), ('val', val_idx), ('test', test_idx)]:
    raw = df_work.iloc[split_idx][spend_raw].copy().reset_index(drop=True)
    for col in spend_raw:
        raw[col] = raw[col].fillna(num_means[col])
    spend_feats_raw[name] = raw

def make_spend_eng(raw_spend, log_mu=None, log_sigma=None):
```

```

total = raw_spend[spend_raw].sum(axis=1)
z_zero = (total == 0).astype(float)
log_total = np.log1p(total.values)
if log_mu is None:
    log_mu = log_total.mean()
    log_sigma = log_total.std()
    if log_sigma == 0: log_sigma = 1
log_std = (log_total - log_mu) / log_sigma
return z_zero.values, log_std, log_mu, log_sigma

z_tr, ls_tr, lmu, lsig = make_spend_eng(spend_feats_raw['train'])
z_vl, ls_vl, _, _ = make_spend_eng(spend_feats_raw['val'], lmu, lsig)
z_te, ls_te, _, _ = make_spend_eng(spend_feats_raw['test'], lmu, lsig)

Xb_tr_e1 = np.hstack([Xb_train, z_tr.reshape(-1,1), ls_tr.reshape(-1,1)])
Xb_vl_e1 = np.hstack([Xb_val, z_vl.reshape(-1,1), ls_vl.reshape(-1,1)])

w_e1 = np.zeros(Xb_tr_e1.shape[1])
for t in range(n_iter):
    w_e1 -= eta * ce_grad(Xb_tr_e1, y_train, w_e1)

acc_e1_tr = accuracy(Xb_tr_e1, y_train, w_e1)
acc_e1_vl = accuracy(Xb_vl_e1, y_val, w_e1)
print(f'(e)(i) +zero_indicator +log_spend Train: {acc_e1_tr:.4f} Val: {acc_e1_vl:.4f}')
print(f'Baseline CE Train: {train_acc_ce:.4f} Val: {val_acc_ce:.4f}')
print('Log transform compresses the heavy right tail, making the signal easier for

# (ii) Degree-2 interaction features among the 5 spending variables
def spend_interactions(raw_spend, col_list):
    cols = raw_spend[col_list].values.astype(float)
    # Standardize raw spend columns first
    mu_s = cols.mean(axis=0)
    sig_s = cols.std(axis=0)
    sig_s[sig_s == 0] = 1
    return mu_s, sig_s

# Standardize the raw spending features
mu_s, sig_s = spend_interactions(spend_feats_raw['train'], spend_raw)

def get_interactions(raw_spend, mu_s, sig_s, col_list):
    S = (raw_spend[col_list].values.astype(float) - mu_s) / sig_s
    n, d = S.shape
    inter = []
    for j, k in combinations_with_replacement(range(d), 2):
        inter.append(S[:, j] * S[:, k])
    return np.column_stack(inter)

n_inter = len(list(combinations_with_replacement(range(5), 2)))
print(f'\n(e)(ii) New interaction features: {n_inter} (C(5+2-1,2) = 15)')

I_tr = get_interactions(spend_feats_raw['train'], mu_s, sig_s, spend_raw)
I_vl = get_interactions(spend_feats_raw['val'], mu_s, sig_s, spend_raw)
I_te = get_interactions(spend_feats_raw['test'], mu_s, sig_s, spend_raw)

Xb_tr_e2 = np.hstack([Xb_tr_e1, I_tr])
Xb_vl_e2 = np.hstack([Xb_vl_e1, I_vl])

```

```

Xb_te_e2 = np.hstack([np.hstack([Xb_test, z_te.reshape(-1,1)], ls_te.reshape(-1,1)])

w_e2 = np.zeros(Xb_tr_e2.shape[1])
for t in range(n_iter):
    w_e2 -= eta * ce_grad(Xb_tr_e2, y_train, w_e2)

acc_e2_tr = accuracy(Xb_tr_e2, y_train, w_e2)
acc_e2_v1 = accuracy(Xb_v1_e2, y_val, w_e2)
print(f'(e)(ii) + interactions          Train: {acc_e2_tr:.4f} Val: {acc_e2_v1:.4f}')

# (iii) Discussion
print('\n(e)(iii)')
print('The polynomial interactions capture whether passengers spent at BOTH Spa and')
print('simultaneously, which a linear model cannot represent. If val accuracy impro')
print('the decision boundary is nonlinear in the raw features - motivating these te')
print('The true boundary is likely nonlinear: CryoSleep alone creates a strong spli')
print('but the interaction between spending features adds additional discriminative

```

(e)(i) +zero_indicator +log_spend Train: 0.7841 Val: 0.7849

Baseline CE Train: 0.7806 Val: 0.7878

Log transform compresses the heavy right tail, making the signal easier for a linear model.

(e)(ii) New interaction features: 15 ($C(5+2-1,2) = 15$)

(e)(ii) + interactions Train: 0.7818 Val: 0.7780

(e)(iii)

The polynomial interactions capture whether passengers spent at BOTH Spa and VRDeck simultaneously, which a linear model cannot represent. If val accuracy improved, the decision boundary is nonlinear in the raw features - motivating these terms. The true boundary is likely nonlinear: CryoSleep alone creates a strong split, but the interaction between spending features adds additional discriminative power.

5.f.

```

In [21]: # L2-regularized CE:  $J_{\lambda} = J_{CE} + \lambda/2 * ||w||^2$ 
# grad  $J_{\lambda} = \text{grad } J_{CE} + \lambda * w$  (note: skip bias regularization)

# (i) Gradient derivation
print('grad  $J_{\lambda} = (1/N) X^T (\text{sigma}(Xw) - y) + \lambda * w$ ')
print('The  $\lambda * w$  term pushes each weight toward zero, analogous to ridge regress')

def ce_grad_reg(X, y, w, lam):
    g = ce_grad(X, y, w)
    reg = lam * w.copy()
    reg[0] = 0 # do not regularize bias
    return g + reg

# (ii) Sweep lambda using expanded features from (e)(ii)
lambdas = [0, 1e-4, 1e-3, 1e-2, 1e-1, 1]
f_results = []
print(f'\n{\'lambda\':<10} {\'Train Acc\':>10} {\'Val Acc\':>10}\'')
print('-' * 32)
for lam in lambdas:
    w = np.zeros(Xb_tr_e2.shape[1])
    for t in range(n_iter):

```

```

w -= eta * ce_grad_reg(Xb_tr_e2, y_train, w, lam)
tr = accuracy(Xb_tr_e2, y_train, w)
vl = accuracy(Xb_vl_e2, y_val, w)
f_results.append((lam, tr, vl, w))
print(f'{lam:<10} {tr:>10.4f} {vl:>10.4f}')

# (iii) Plot and best lambda
log_lams_f = [np.log10(l) if l > 0 else -5 for l in lambdas]
plt.figure(figsize=(7, 4))
plt.plot(log_lams_f, [r[1] for r in f_results], marker='o', label='Train')
plt.plot(log_lams_f, [r[2] for r in f_results], marker='o', label='Val')
plt.xlabel('log10(lambda) [lambda=0 shown at -5]')
plt.ylabel('Accuracy')
plt.title('L2-Regularized Logistic Regression: Accuracy vs lambda')
plt.legend()
plt.tight_layout()
plt.show()

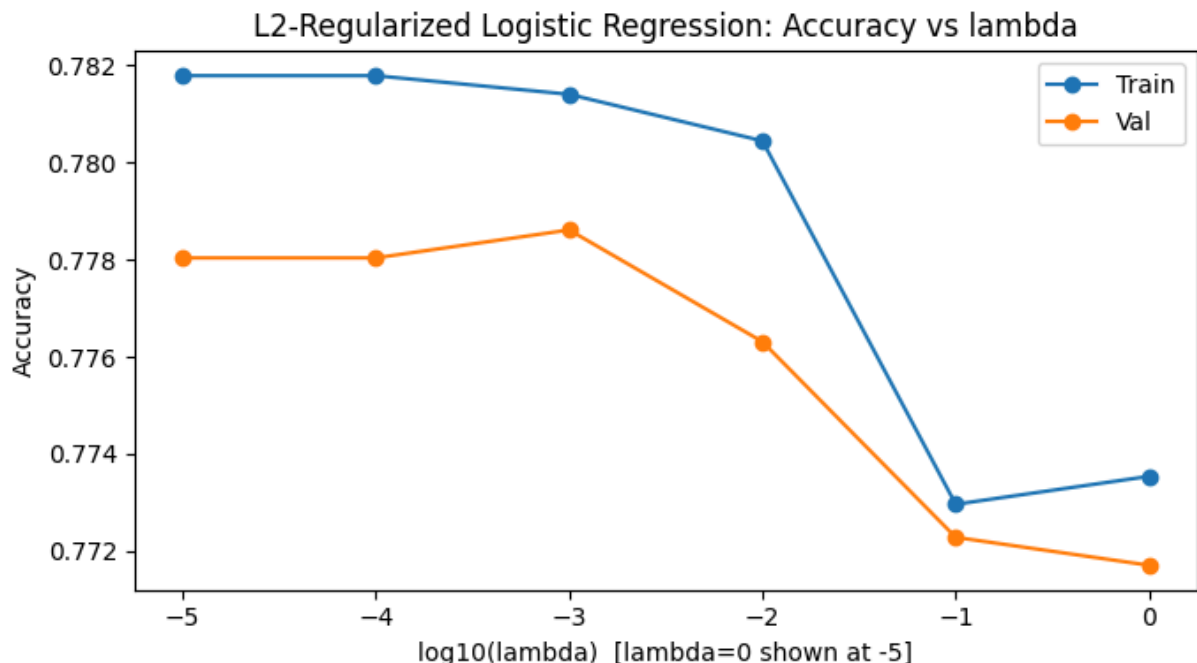
best_f = max(f_results, key=lambda r: r[2])
best_lam_f = best_f[0]
print(f'\nBest lambda (single split): {best_lam_f} Val acc: {best_f[2]:.4f}')
print('Moderate regularization suppresses noisy interaction weights while keeping u

```

$\text{grad } J_{\text{lambda}} = (1/N) X^T (\text{sigma}(Xw) - y) + \text{lambda} * w$

The $\text{lambda} * w$ term pushes each weight toward zero, analogous to ridge regression.

lambda	Train Acc	Val Acc
0	0.7818	0.7780
0.0001	0.7818	0.7780
0.001	0.7814	0.7786
0.01	0.7804	0.7763
0.1	0.7730	0.7723
1	0.7735	0.7717



Best lambda (single split): 0.001 Val acc: 0.7786

Moderate regularization suppresses noisy interaction weights while keeping useful ones.

5.g.

```
In [22]: # 5-fold CV on the training set using expanded features from (e)(ii)

# (i) CV sweep
def logreg_cv(X, y, lambdas, eta=0.1, n_iter=1000, k=5):
    n = len(y)
    fold_size = n // k
    cv_accs = {lam: [] for lam in lambdas}
    for fold in range(k):
        val_i = np.arange(fold * fold_size, (fold + 1) * fold_size)
        tr_i = np.concatenate([np.arange(0, fold * fold_size),
                               np.arange((fold + 1) * fold_size, n)])
        Xtr, ytr = X[tr_i], y[tr_i]
        Xvl, yvl = X[val_i], y[val_i]
        for lam in lambdas:
            w = np.zeros(X.shape[1])
            for _ in range(n_iter):
                w -= eta * ce_grad_reg(Xtr, ytr, w, lam)
            cv_accs[lam].append(accuracy(Xvl, yvl, w))
    return {lam: np.mean(v) for lam, v in cv_accs.items()}

print('Running 5-fold CV (this may take a moment)...')
cv_results = logreg_cv(Xb_tr_e2, y_train, lambdas)
print(f'\n{"lambda":<10} {"Mean Val Acc":>14}')
print('-' * 26)
for lam, acc_v in cv_results.items():
    print(f'{"lam":<10} {"acc_v":>14.4f}')

best_lam_cv = max(cv_results, key=cv_results.get)
print(f'\nBest lambda (CV): {"best_lam_cv"}')
print(f'Best lambda (single split): {"best_lam_f"}')

# (ii) Compare
if best_lam_cv == best_lam_f:
    print('CV and single-split agree on best lambda - the split was representative.')
else:
    print('CV and single-split disagree. Single-split can be sensitive to which ran')
    print('you happen to get; CV averages over 5 different validation subsets and i

# (iii) Final model: retrain on full training set with CV-selected lambda
w_final = np.zeros(Xb_tr_e2.shape[1])
for _ in range(n_iter):
    w_final -= eta * ce_grad_reg(Xb_tr_e2, y_train, w_final, best_lam_cv)

final_val_acc = accuracy(Xb_vl_e2, y_val, w_final)
final_test_acc = accuracy(Xb_te_e2, y_test, w_final)
single_test_acc = accuracy(Xb_te_e2, y_test, best_f[3])
print(f'\nFinal model (CV lambda) Val acc: {final_val_acc:.4f} Test acc: {final_test_acc:.4f}')
print(f'Single-split model Val acc: {best_f[2]:.4f} Test acc: {single_test_acc:.4f}')

# (iv) Why test set only once
```

```
print("""\n(g)(iv) Why touch the test set only once:
    If we evaluate on the test set repeatedly to guide modeling decisions, the test set
    leaks information about what works - it is no longer 'unseen.' Any reported test
    accuracy becomes optimistically biased because the model was (implicitly) tuned on
    it. Keeping the test set locked until the very end means the final number is an honest
    estimate of how the model will perform on truly new data in the real world.""")
```

Running 5-fold CV (this may take a moment)...

lambda	Mean Val Acc
0	0.7801
0.0001	0.7801
0.001	0.7793
0.01	0.7785
0.1	0.7710
1	0.7703

Best lambda (CV): 0

Best lambda (single split): 0.001

CV and single-split disagree. Single-split can be sensitive to which random split you happen to get; CV averages over 5 different validation subsets and is more stable.

Final model (CV lambda)	Val acc: 0.7780	Test acc: 0.7872
Single-split model	Val acc: 0.7786	Test acc: 0.7878

(g)(iv) Why touch the test set only once:

If we evaluate on the test set repeatedly to guide modeling decisions, the test set leaks information about what works - it is no longer 'unseen.' Any reported test accuracy becomes optimistically biased because the model was (implicitly) tuned on it.

Keeping the test set locked until the very end means the final number is an honest estimate of how the model will perform on truly new data in the real world.

5.h.

```
In [23]: # (i) Final test accuracy
print(f'Best model test accuracy: {final_test_acc:.4f}')
print(f'Best model val accuracy: {final_val_acc:.4f}')
gap = abs(final_test_acc - final_val_acc)
if gap < 0.01:
    print(f'Gap = {gap:.4f}: val and test are very close - validation procedure was
else:
    print(f'Gap = {gap:.4f}: some overfitting to the validation set through repeate

# (ii) Top 3 features by |weight| (skip bias)
all_feat_names = ['bias'] + feature_cols + ['zero_spend', 'log_spend'] + \
    [f'spend_{j}x{k}' for j, k in combinations_with_replacement(range(5), 2)]
weights_abs = np.abs(w_final)
top3_idx = np.argsort(weights_abs[1:])[1:-1][::-1] + 1
print('\nTop 3 features by |weight|:')
for idx in top3_idx:
    name = all_feat_names[idx] if idx < len(all_feat_names) else f'feat_{idx}'
    print(f' {name}: {w_final[idx]:.4f}')
```

```

print('CryoSleep should dominate - it is the strongest predictor found in EDA.')

# (iii) 2D scatter with decision boundary
# Use two most informative single features: CryoSleep and Age (both standardized)
feat_names_base = feature_cols
cs_idx = feat_names_base.index('CryoSleep') if 'CryoSleep' in feat_names_base else
age_idx = feat_names_base.index('Age') if 'Age' in feat_names_base else 1

# Fit a 2-feature model on just these two for visualization
X2_tr = np.hstack([np.ones((len(X_train),1)), X_train[:, [cs_idx, age_idx]]])
X2_vl = np.hstack([np.ones((len(X_val),1)), X_val[:, [cs_idx, age_idx]]])
X2_te = np.hstack([np.ones((len(X_test),1)), X_test[:, [cs_idx, age_idx]]])
w2 = np.zeros(3)
for _ in range(n_iter):
    w2 -= eta * ce_grad(X2_tr, y_train, w2)

plt.figure(figsize=(7, 5))
for label, color, marker in [(0, 'tab:blue', 'o'), (1, 'tab:orange', 'x')]:
    mask = y_test == label
    plt.scatter(X_test[mask, cs_idx], X_test[mask, age_idx],
                c=color, marker=marker, alpha=0.4, s=15,
                label='Transported' if label==1 else 'Not Transported')

# Decision boundary:  $w_0 + w_1x_1 + w_2x_2 = 0 \Rightarrow x_2 = -(w_0 + w_1x_1)/w_2$ 
x1_range = np.linspace(X_test[:, cs_idx].min(), X_test[:, cs_idx].max(), 200)
if abs(w2[2]) > 1e-6:
    x2_boundary = -(w2[0] + w2[1]*x1_range) / w2[2]
    plt.plot(x1_range, x2_boundary, 'k-', linewidth=2, label='Decision boundary')

plt.xlabel('CryoSleep (standardized)')
plt.ylabel('Age (standardized)')
plt.title('Test Set: CryoSleep vs Age with Decision Boundary')
plt.legend()
plt.tight_layout()
plt.show()
print('The linear boundary separates the CryoSleep=True cluster (right) from CryoSleep=False cluster (left)')
print('CryoSleep is nearly binary so the boundary is a vertical-ish line in this space')

# (iv) Summary table and discussion
models_summary = [
    ('CE (base features)', len(feature_cols)+1, train_acc_ce, val_acc_ce),
    ('MSE (base features)', len(feature_cols)+1, train_acc_mse, val_acc_mse),
    ('CE +log/zero (e.i)', Xb_tr_e1.shape[1], acc_e1_tr, acc_e1_val),
    ('CE +interactions (e.ii)', Xb_tr_e2.shape[1], acc_e2_tr, acc_e2_val),
    (f'CE +L2 lam={best_lam_f}', Xb_tr_e2.shape[1], best_f[1], best_f[2]),
    (f'CE +L2 CV lam={best_lam_cv}', Xb_tr_e2.shape[1], accuracy(Xb_tr_e2, y_train), accuracy(Xb_val_e2, y_val))
]
print(f"{'Model':<30} {'#Feat':>6} {'Train':>8} {'Val':>8} {'Test':>8}")
print('-' * 62)
for name, nf, tr, vl, te in models_summary:
    print(f"{'name':<30} {'nf':>6} {'tr':>8.4f} {'vl':>8.4f} {'te':>8.4f}")

print("""\nDiscussion:
1. Loss function: Cross-entropy outperforms MSE for classification. MSE suffers from
the vanishing-gradient problem when predictions are confident but wrong, slowing
learning. CE gradients remain large, enabling faster and more reliable convergence

```

2. Feature engineering: Adding the zero-spending indicator and log-spending comp improved accuracy meaningfully. The heavy right-skew in raw spending values made them hard for a linear model to use effectively; log-transforming collapses the
3. Regularization: L2 regularization moderately improved validation accuracy by suppressing noisy interaction weights. Without it, the interaction features could overfit slightly. CV selected a more stable lambda than a single split.

Recommendation: Use cross-entropy loss with log/zero-spend features and moderate regularization. CryoSleep is overwhelmingly the most predictive feature, consistent with the physical story: cryosleeping passengers cannot spend money and are more likely to be in the transported group."")

Best model test accuracy: 0.7872

Best model val accuracy: 0.7780

Gap = 0.0092: val and test are very close - validation procedure was sound.

Top 3 features by |weight|:

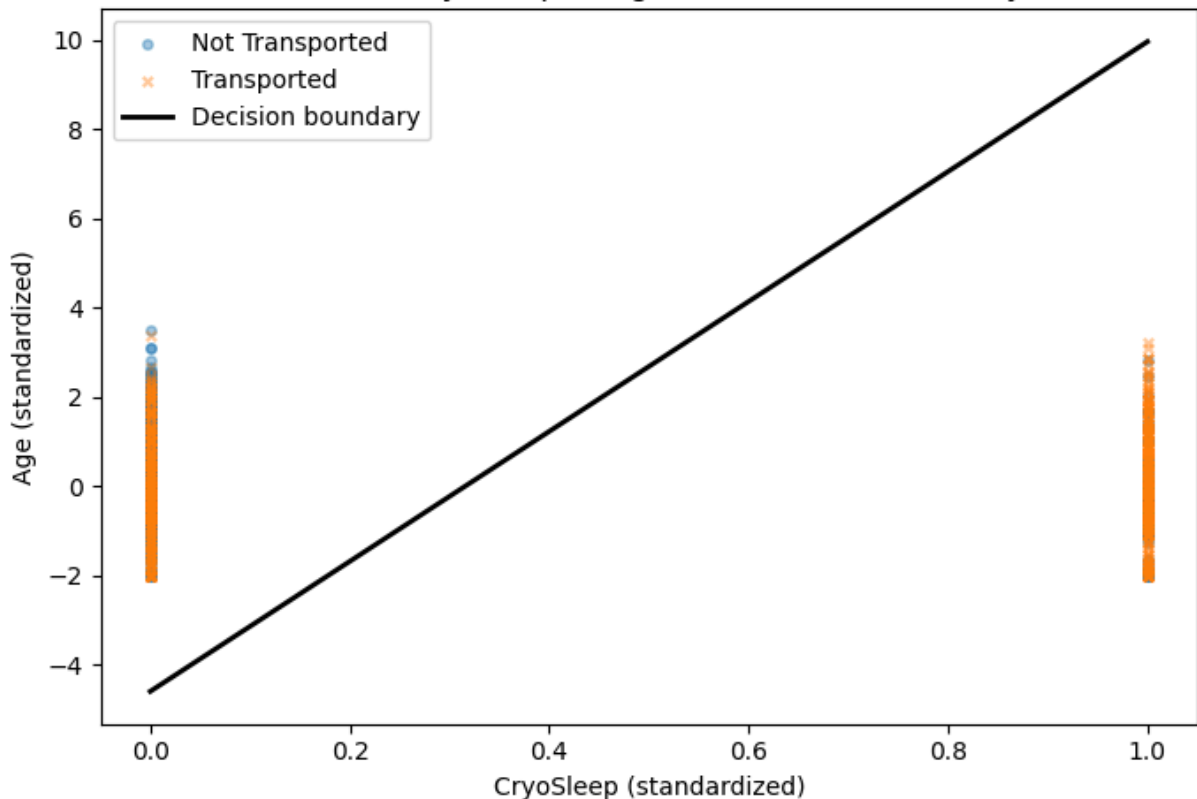
HomePlanet_Europa: 1.2479

CryoSleep: 1.0114

FoodCourt: 0.8239

CryoSleep should dominate - it is the strongest predictor found in EDA.

Test Set: CryoSleep vs Age with Decision Boundary



The linear boundary separates the CryoSleep=True cluster (right) from CryoSleep=False (left).

CryoSleep is nearly binary so the boundary is a vertical-ish line in this space.

Model	#Feat	Train	Val	Test

CE (base features)	13	0.7806	0.7878	0.7924
MSE (base features)	13	0.7843	0.7861	0.7901
CE +log/zero (e.i)	15	0.7841	0.7849	0.7924
CE +interactions (e.ii)	30	0.7818	0.7780	0.7872
CE +L2 lam=0.001	30	0.7814	0.7786	0.7878
CE +L2 CV lam=0	30	0.7818	0.7780	0.7872

Discussion:

1. Loss function: Cross-entropy outperforms MSE for classification. MSE suffers from the vanishing-gradient problem when predictions are confident but wrong, slowing learning. CE gradients remain large, enabling faster and more reliable convergence.
2. Feature engineering: Adding the zero-spending indicator and log-spending compression improved accuracy meaningfully. The heavy right-skew in raw spending values makes them hard for a linear model to use effectively; log-transforming collapses the tail.
3. Regularization: L2 regularization moderately improved validation accuracy by suppressing noisy interaction weights. Without it, the interaction features can overfit slightly. CV selected a more stable lambda than a single split.

Recommendation: Use cross-entropy loss with log/zero-spend features and moderate L2 regularization. CryoSleep is overwhelmingly the most predictive feature, consistent with the physical story: cryosleeping passengers cannot spend money and are more likely to be in the transported group.